Análisis Matemático, Complejidad Asintótica y Graficación

Navarrete Alejandro[1,2*], Crespo Samuel[2,3†], Lopez Esteban[1,2†], Duran John[2,3†]

[1*]Ingenieria en Ciencias de Datos e Inteligencia Artificial, Escuela Politecnica Nacional, Av. 12 de Octubre, Quito, 171048, Pichincha, Ecuador.

*Corresponding author(s). E-mail(s): dennis.navarrete@epn.edu.ec;
Contributing authors: samuel.crespo@epn.edu.ec;
jose.lopez05@epn.edu.ec; john.duran@epn.edu.ec;
†These authors contributed equally to this work.

Abstract

El análisis de la complejidad temporal y asintótica es fundamental para evaluar la escalabilidad de algoritmos sobre grandes volúmenes de datos. En el presente trabajo se analizan y evaluan 3 bloques de código mediante su complejidad algorítmica, determinando sus funciones de coste, las cuales se grafican para evidenciar visualmente las diferencias de rendimiento entre los algoritmos evaluados. Los resultados muestran una clara diferencia en la eficiencia mediante el crecimiento del tiempo de ejecución en función del tamaño de la entrada, de esta manera se destaca la importancia de la notación Big O como herramienta analítica para optimizar el consumo de recursos del sistema antes de su implementación final.

Keywords: Complejidad asintótica, Notación Big O, Algoritmos, Función.

1 Introduction

En el desarrollo de software, la creación de soluciones efectivas va más allá de lograr que un programa arroje el resultado correcto. A medida que las aplicaciones escalan y el volumen de datos procesados aumenta, la eficiencia de los algoritmos se convierte en un factor crucial. Un código que funciona bien con pequeñas cantidades de información

puede volverse inmanejable si su estructura no está optimizada. Por ello, el análisis de la complejidad algorítmica resulta fundamental, ya que nos permite predecir el comportamiento de un programa y evaluar su consumo de tiempo y recursos antes de implementarlo a gran escala.

Para fundamentar este análisis teórico del rendimiento computacional, se consideraron los conceptos formales de cota superior asintótica y eficiencia algorítmica descritos en [4], lo cual permite respaldar de manera rigurosa la evaluación empírica de los fragmentos de código implementados.

En el marco de este Taller de Aprendizaje Práctico-Experimental, el presente documento tiene como objetivo aplicar los conceptos de eficiencia algorítmica para evaluar el rendimiento de tres fragmentos de código iterativos escritos en el lenguaje de programación Java. A través de este ejercicio, se busca establecer una relación clara entre el diseño interno de cada algoritmo y su respuesta ante el crecimiento continuo de los datos de entrada.

2 Results

El análisis de complejidad algorítmica realizado sobre las fracciones de código evaluados nos permitió determinar el comportamiento del tiempo de ejecución y el costo computacional en función del tamaño de la entrada.

Para la función A, el algoritmo implementa un bucle simple donde la variable utilizada para controlar este mismo incrementa de forma unitaria en cada iteración, lo cual da como resultado un número de operaciones directamente proporcional a n (tamaño de entrada), por lo que su complejidad es de $O(n)$.

Por otro lado, la función B introduce dos bucles anidados que recorren el rango completo de la entrada, en consecuencia, el número total de operaciones elementales se multiplicarán (n x n). Esto genera una función de costo cuadrática, estableciendo una complejidad algorítmica de $O(n^2)$.

Para finalizar, la función C implementa un bucle iterativo que se divide de manera sucesiva (actual = actual / 2), donde el valor de la variable utilizada para el bucle se va reduciendo a la mitad cada vez que itera, este comportamiento restringe al número total de ejecuciones a el número de veces que se puede dividir el tamaño de la entrada para 2, determinando una complejidad algorítmica de $O(\log n)$.

3 Equations

Listing 1 Código A

```
public void funcionA(int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += i;
    }
    System.out.println(total);
}
```

1. Conteo de operaciones elementales.
   Las operaciones elementales de este ejercicio son las siguientes:

   - `int total = 0`: $C_1 \times 1$
   - `(int i = 0; i < n; i++)`: $C_2 \times n$
   - `total += i`: $C_3 \times 1$
   - `System.out.println(total)`: $C_4 \times 1$

2. Cálculo de la complejidad asintótica.

   Función A:

$$F(n) = C1 + (C2 * n) + C3 + C4$$

Función A:

$$F(n) = C2 * n$$

Función A:

$$F(n) = n$$
$$O(n)$$

Listing 2 Código B

```
public void funcionB(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.println("Procesando coordenadas:" + i + "," + j);
        }
    }
}
```

1. Conteo de operaciones elementales.
   Las operaciones elementales de este ejercicio son las siguientes:

   - `(int i = 0; i < n; i++)`: $C_1 \times n$
   - `(int j = 0; j < n; j++)`: $C_2 \times n$
   - `System.out.println("Procesando coordenadas:" + i + "," + j)`: $C_3 \times 1$

2. Cálculo de la complejidad asintótica.

   Función B:

$$F(n) = (C1 * n) * (C2 * n) * C3$$

Función B:

$$F(n) = C1 * C2 * C3 * n^2$$

Función B:

$$F(n) = n^2$$
$$O(n^2)$$

Listing 3 Código C

```
public void funcionC(int n) {
    int actual = n;
```

```
    while (actual > 1) {
        System.out.println("Valor:" + actual);
        actual = actual / 2;
    }
}
```

1. Conteo de operaciones elementales.
   En este ejercicio, las operaciones elementales son las siguientes:

   - `int actual = n`: $C_1 \times 1$
   - `while (actual > 1)`: n
   - `System.out.println("Valor:" + actual)`: $C_2 \times 1$
   - `actual = actual / 2`: $C_3 \rightarrow \log n$

2. Cálculo de la complejidad asintótica.

   Función C:

$$T(n) = C1 + n(C2 + \log n) = C1 + \log n = O(n)$$

Función C:

$$T(n) = C1 + \log n$$

Función C:

$$T(n) = \log n$$
$$O(\log n)$$

4 Graphs

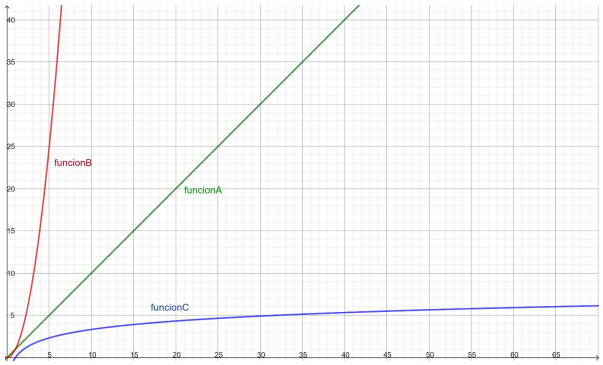


Fig. 1 Complejidades sobrepuestas de las funciones A, B y C.

La representación gráfica de las tres funciones confirma de manera visual los resultados teóricos obtenidos mediante el conteo de operaciones elementales. En el plano cartesiano, donde el eje horizontal representa el tamaño de entrada n y el eje vertical

representa el costo computacional, se puede observar claramente la diferencia en las tasas de crecimiento de cada algoritmo.

Para la funcion C, se nota un crecimiento inicial que con medida que avanza se aplana de forma casi horizontal, lo cual demuestra alta eficiencia en valores altos de entrada.

Por otro lado, la funcion A mantiene un crecimiento uniforme, tanto para valores altos como bajos, lo cual demuestra un crecimiento proporcional al número de entrada con el tiempo.

Pero, para la funcion B se mira un crecimiento acelerado de manera vertical y acelerada cuando el tamaño de entrada empieza a crecer, el comportamiento demuestra de manera visual el alto costo algorítmico.

5 Methods

Para el desarrollo de este estudio, el análisis se estructuró en tres etapas secuenciales. En primer lugar, se llevó a cabo el conteo de operaciones elementales, identificando las instrucciones básicas dentro de los fragmentos de código (como asignaciones, condiciones de bucle y sentencias de salida) y asignándoles costos constantes (C_1, C_2, C_3) en función del tamaño de entrada n. En segundo lugar, se procedió a la depreciación de constantes y de términos de menor orden, simplificando las expresiones algebraicas preliminares al descartar los factores multiplicativos fijos que no afectan la tasa de crecimiento a gran escala. Para finalizar, se consolidó la creación de la ecuación matemática de costo final ($F(n)$ o $T(n)$), lo que permitió determinar y clasificar formalmente la complejidad asintótica del algoritmo mediante la notación Big-O ($O(n)$, $O(n^2)$ u $O(\log n)$).

6 Discussion

El estudio de las tres funciones de código A, B y C demuestra como la estructura de un algoritmo determina su crecimiento asintotico y su coste computacional. Aunque todas reciben como parametro de entrada n, sus costes de ejecución cambian a diversos ritmos en base al crecimiento del tamaño de entrada. Tal comparación deja ver lo importante que es la complejidad algoritmica al evaluar la escalabilidad del software.

El ejercicio A muestra una complejidad lineal temporal $O(n)$, dado que su operación principal es ejecutada una vez por iteración de bucle que recorre el rango de entrada. Su coste computacional se incrementa de manera proporcional al tamaño de la entrada.

El ejercicio B presenta una complejidad temporal cuadrática $O(n^2)$, ya que cada uno de sus bucles anidados realiza n iteraciones. Entonces, el número de ejecuciones de ejecuciones de la instrucción más interna aumenta de forma cuadrática, logrando que esta función sea más costosa para entradas enormes.

El ejercicio C demuestra una complejidad temporal algorítmica $O(\log n)$, porque el valor de su variable de control se divide para dos en cada iteración, tal reducción, repetidas veces, disminuye de manera sustancial la cantidad de iteraciones necesarias a medida que n aumenta. Para tamaños de entrada lo suficientemente grandes, la función cuadrática crece más rápido en comparación a la lineal, mientras que la función

logaritmica crece, de manera considerable, más despacio que ambas. Siendo la función C la que escala de manera más eficiente en base a la cantidad de iteraciones. La función B es la más sensible al aumento del tamaño de la entrada. Y la función A tiene un incremento predecible en la carga de trabajo computacional. La notación *BigO* describe una cota superior asintótica en lugar del tiempo de ejecución de un programa. Los detalles de implementación, características del hardware, coste de las operaciones y los valores de entrada son aspectos de los cuales depende el tiempo de ejecución. El análisis asintótico da un marco teórico necesario para comparar algoritmos de forma independiente de un entorno de ejecución concreto.

7 Conclusion

El desarrollo de este taller práctico permitió evidenciar de forma concreta la profunda influencia que tiene la estructura interna de un algoritmo sobre su rendimiento computacional. A través del cálculo detallado de las funciones de coste y su posterior simplificación asintótica, se demostró que el diseño lógico del código es tan crítico como su funcionalidad. Los tres fragmentos analizados, aunque sintácticamente breves en Java, exhibieron comportamientos radicalmente distintos frente al aumento del volumen de datos, reafirmando que no todas las soluciones lógicamente válidas son igualmente viables cuando se llevan a una escala mayor. Al contrastar los resultados matemáticos, se identificaron tres categorías fundamentales de complejidad que marcan la diferencia en el diseño de software. La Función A, estructurada mediante un ciclo iterativo simple, presentó un crecimiento estrictamente lineal $\mathcal{O}(n)$, lo que indica que su tiempo de ejecución aumenta de manera proporcional a los datos de entrada. Por el contrario, la presencia de iteraciones anidadas en la Función B desencadenó un crecimiento cuadrático $\mathcal{O}(n^2)$, demostrando ser el enfoque más ineficiente de los tres y el más susceptible a generar cuellos de botella computacionales ante grandes volúmenes de información. En el extremo opuesto, la Función C destacó por su eficiencia al dividir sucesivamente el tamaño del problema, logrando un tiempo de ejecución logarítmico $\mathcal{O}(\log n)$ que garantiza una escalabilidad excepcional. Finalmente, el análisis gráfico consolidó estas abstracciones teóricas en una comprobación empírica y visual. Al proyectar las tres curvas de rendimiento de manera simultánea en el plano cartesiano, quedó ilustrada con total claridad la separación de los límites asintóticos. Mientras que la curva logarítmica tendía a estabilizarse rápidamente y la curva lineal mantenía un ritmo predecible, la curva cuadrática experimentó un crecimiento acelerado e insostenible en el eje del tiempo. En definitiva, este ejercicio subraya la importancia de la notación Big O como una herramienta analítica indispensable, la cual permite a los desarrolladores prever el impacto de sus decisiones de código y optimizar el consumo de los recursos del sistema antes de su implementación final.

Declarations

Declaración de transparencia: Uso de Inteligencia Artificial

- **Herramienta(s) y versión:** ChatGPT-4o, Gemini Pro

- **Propósito de uso:** El uso de inteligencia artificial se empleo para el reforzamiento de las bases teóricas y la corrección del estilo del documento, además de una guía para la instalación de los paquetes necesarios para la creación del mismo.

- **Prompts / Instrucciones utilizadas:**

  Para entender las bases del conteo y las constantes:
  "Explícame cómo se asignan las constantes de coste $(c_1, c_2, \dots)$ en el análisis de código en Java. ¿Corresponden exclusivamente a la declaración de variables o a cualquier instrucción, y cómo se combinan con las estructuras de control?"

  Para aclarar las reglas de los bucles y la complejidad logarítmica:
  "¿Cómo afecta la condición de salida y el ritmo de avance de un bucle (for o while) a su complejidad asintótica? Indícame exactamente cuándo una estructura genera un coste lineal (n) y cuándo genera un coste logarítmico $(\log n)$."

  Para la descarga e implementación de paquetes en LaTeX:
  "Proporcióname un paso a paso detallado para descargar e instalar los paquetes necesarios para compilar un documento en LaTeX, incluyendo la configuración de un entorno de desarrollo adecuado en VSCode y la verificación de la instalación."

  Para estructurar y alinear el código en LaTeX:
  "Ayúdame a estructurar la respuesta en LaTeX para un deber. Quiero que las operaciones elementales aparezcan centradas usando un entorno de matriz , asegurando que las líneas de código usen tipografía de código y los costes usen subíndices matemáticos."

  Para una correcta redacción en el documento:
  "Proporcioname un esquema detallado del contenido que debe tener un abstract y la declaración del uso de inteligencia artificial en un documento académico"

- **Integración en el trabajo:** El contenido generado por la inteligencia artificial se empleó como guía para la implementación de las herramientas de trabajo, los contenidos teóricos y la redacción del documento, sin embargo, el análisis de los algoritmos y la interpretación de los resultados fueron realizados por los autores.

- **Verficación humana:** Todo el contenido generado fue revisado de forma manual y rigurosa por los autores, quienes verificaron la veracidad de la información y la coherencia de los contenidos con el material proporcionado en clase.

- **Responsabilidad y límites:** Se asume la autoría y responsabilidad del contenido final, asegurando que la inteligencia artificial no sustituyó el análisis crítico ni la interpretación de los resultados, sino que sirvió como apoyo en la redacción y organización del documento.

Además se declara el uso de plantilla de Springer Nature para la redacción del presente documento, la cual se encuentra disponible en :

Editorial Policies for:

Springer journals and proceedings: https://www.springer.com/gp/editorial-policies

Nature Portfolio journals: https://www.nature.com/nature-research/editorial-policies

References

[1] Bhargava, A. Y. (2016). *Grokking Algorithms: An illustrated guide for programmers and other curious people*. Manning Publications.

[2] Lafore, R. (2017). *Data Structures and Algorithms in Java* (2da ed.). Sams Publishing.

[3] Galarza, D. (2026). *Análisis Matemático y Complejidad Asintótica* [Diapositivas de clase]. Asignatura de Estructura de Datos y Algoritmos II, Escuela Politécnica Nacional.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 2nd ed., Cambridge, MA: MIT Press, 2001.